

UNIT 3

Scenario 1: Why Clean Data?

Problem:

A dataset contains missing values and inconsistent data formats. We need to clean it before analysis.

```
python
import pandas as pd

# Sample raw data
data = {
    "Name": ["Alice", "Bob", None, "Charlie", "David"],
    "Age": [25, "Thirty", 29, None, 22],
    "Salary": [50000, 55000, 60000, 58000, "Sixty-Five Thousand"]
}

df = pd.DataFrame(data)

# Display raw data
print("Raw Data:")
print(df)

# Handling missing values and standardizing data formats
df["Name"].fillna("Unknown", inplace=True)
df["Age"] = pd.to_numeric(df["Age"], errors="coerce") # Convert Age to numeric
df["Salary"] = df["Salary"].replace("Sixty-Five Thousand",
65000).astype(int) # Fix salary format

# Display cleaned data
print("\nCleaned Data:")
print(df)
```

Output:

```
... Raw Data:
   Name  Age  Salary
0  Alice   25   50000
1   Bob  Thirty   55000
2   None   29   60000
3  Charlie  None   58000
4   David   22  Sixty-Five Thousand

Cleaned Data:
   Name  Age  Salary
0  Alice  25.0   50000
1   Bob  NaN   55000
2  Unknown  29.0   60000
3  Charlie  NaN   58000
4   David  22.0   65000

/tmp/ipykernel_1425/2678620554.py:17: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the op

df["Name"].fillna("Unknown", inplace=True)
/tmp/ipykernel_1425/2678620554.py:19: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the old behavior, expl
df["Salary"] = df["Salary"].replace("Sixty-Five Thousand", 65000).astype(int) # Fix salary format
```

Scenario 2: Identifying Values for Data Cleanup

Problem:

Identifying missing values and incorrect entries in a dataset.

```
# Checking for missing values
print("Missing Values Count:")
print(df.isnull().sum())

# Identifying incorrect entries
print("\nData Types:")
print(df.dtypes)
```

Output:

```
... Missing Values Count:
      Name      0
      Age      2
      Salary    0
      dtype: int64

      Data Types:
      Name      object
      Age      float64
      Salary    int64
      dtype: object
```

Scenario 3: Formatting Data

Problem:

The Date column is in multiple formats. We need to standardize it.

```
data = {'Date': ['2025-01-01', '01-02-2025', '03/05/2025', 'April 7, 2025']}
df = pd.DataFrame(data)

# Converting all date formats to standard YYYY-MM-DD
df['Date'] = pd.to_datetime(df['Date'])

print(df)
```

Output:

```
      Date
0 2025-01-01
1 2025-02-01
2 2025-03-05
3 2025-04-15
```

Scenario 4: Finding Outliers and Bad Data

Problem:

Detecting outliers in the `Sales` column using the IQR method.

```
import numpy as np

data = {'Sales': [200, 250, 300, 1500, 260, 270, 280, 10000]}
df = pd.DataFrame(data)

Q1 = np.percentile(df["Sales"], 25)
Q3 = np.percentile(df["Sales"], 75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Sales"] < lower_bound) | (df["Sales"] > upper_bound)]
print("Outliers:")
print(outliers)
```

Output:

```
• Outliers:
  Sales
3  1500
7 10000
```

Scenario 5: Finding Duplicates

Problem:

Identifying and removing duplicate records in a dataset.

```
data = {'ID': [1, 2, 3, 2, 4, 5, 3], 'Value': [100, 200, 300, 200, 400, 500, 300]}
df = pd.DataFrame(data)

duplicates = df[df.duplicated()]
print("Duplicates Found:")
print(duplicates)

df_cleaned = df.drop_duplicates()
print("\nCleaned Data:")
print(df_cleaned)
```

Output:

```
• Duplicates Found:
  ID  Value
3   2   200
6   3   300

Cleaned Data:
  ID  Value
0   1   100
1   2   200
2   3   300
4   4   400
5   5   500
```

Scenario 6: Fuzzy Matching

Problem:

Matching similar but slightly misspelled names in a dataset.

```
from fuzzywuzzy import fuzz, process

names = ["John Doe", "Jon Doe", "Jane Doe", "J0hn Doe"]
query = "John Doe"

matches = process.Extract(query, names, limit=3)
print("Fuzzy Matches:")
print(matches)
```

```
Requirement already satisfied: fuzzywuzzy in /usr/local/lib/python3.12/dist-packages (0.18.0)
Requirement already satisfied: python-Levenshtein in /usr/local/lib/python3.12/dist-packages (0.27.3)
Requirement already satisfied: Levenshtein==0.27.3 in /usr/local/lib/python3.12/dist-packages (from python-Levenshtein) (0.27.3)
Requirement already satisfied: rapidfuzz<4.0.0,>=3.9.0 in /usr/local/lib/python3.12/dist-packages (from Levenshtein==0.27.3->python-Levenshtein) (3.14.5)
Fuzzy Matches:
[('John Doe', 100), ('Jon Doe', 93), ('J0hn Doe', 88)]
```

Scenario 7: RegEx Matching

Problem:

Finding invalid email formats using regex.

```
import re

emails = ["test@example.com", "user.name@domain.com", "invalid-email@",
"hello@world"]

def is_valid_email(email):
    pattern = r'^[a-zA-Z0-9_+]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$'
    return bool (re.match(pattern, email))

invalid_emails = [email for email in emails if not is_valid_email(email)]
print("Invalid Emails:", invalid_emails)
```

Output:

```
Invalid Emails: ['invalid-email@', 'hello@world']
```

Scenario 8: Normalizing and Standardizing Data

Problem:

Standardizing a column using Min-Max scaling.

```
from sklearn.preprocessing import MinMaxScaler

data = {'Value': [100, 200, 300, 400, 500]}
df = pd.DataFrame(data)
```

```

scaler = MinMaxScaler()
df["Normalized"] = scaler.fit_transform(df[["Value"]])

print(df)

```

OUTPUT

	Value	Normalized
0	100	0.00
1	200	0.25
2	300	0.50
3	400	0.75
4	500	1.00

Scenario 9: Saving the Data

Problem:

Saving the cleaned data to a CSV file.
df.to_csv("cleaned_data.csv", index=False)
print ("Data saved successfully.")

OUTPUT

```
Data saved successfully.
```

Scenario 10: Automating Data Cleanup with a Script

Problem:

Creating a script to clean a dataset and apply transformations.

```

def clean_data(df):
    df.fillna("Unknown", inplace=True)
    df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
    return df

```

```

# Apply function
df = clean_data(df)
print(df)

```

OUTPUT:

	Name	Age
0	Alice	25.0
1	Bob	NaN
2	Unknown	29.0
3	Charlie	NaN